RAJALAKSHMI
ENGINEERING COLLEGE
An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

BONAFIDE CERTIFICATE

NAME MOHAMAD FAYAS N

ACADEMIC YEAR 2026-27 SEMESTER V BRANCH CSD

UNIVERSITY REGISTER No. 2116241701031

Certified that this is the bonafide record of work done by the above student in the Graphics and Multimedia Laboratory during the year 2026 - 2027

Signature of Faculty - in - Charge

Submitted for the Practical Examination held on .....

Internal Examiner

External Examiner

Expno	DATE	TITLE	TR sign
1.	21/7	Bresenham's Line Drawing Algo for |m| < 1	
2.	28/7	Bresenham's Circle Algorithm	
3.	28/7	Bresenham's Ellipse drawing Algorithm	
4.	4/8	DDA Line drawing Algorithm	
5.	4/8	object using Bresenham's Algorithm	
6.	11/8	Rectangle using Bresenham's algorithm	
7)	11/8	traffic signal using mid point Algorithm	
8	11/8	Smiling face using mid point	
9	25/8	Olympic logo using Mid point	
10	25/8	Butterfly using mid point	
11	8/9	Implement Scan conversion of polygon & use flood fill Algorithm	
12)	15/9	Implementation of 2D shape & basic transformation	
13		polygon clipping using Sutherland hodgman Algorithm	
14		create and Render 3D objects using polygons and Apply Basic color and Shading technique	
15		Create multimedia App that integrates img, Sound, video in simple user Interface	
16		3D transformation on basic Transformation	
17		Capture video/audio from webcam on multimedia Interface	
18		Create an application to send & receive multimedia messages (Text, image, audio)	

Exp: 7

Date: 21/7

BRESENHAM'S LINE DRAWING ALGORITHM for |m| < 1

Aim:

To implement Bresenham's Line drawing algorithm for |m| < 1 using Python programming

Program:

```
import matplotlib.pyplot as plt
def bresenham_line (x, y, x2, y2):
    points = [ ]
    dx = x2 - x1
    dy = y2 - y1

    p = 2 * dy - dx
    x = x1
    y = y1
    points.append ((x,y))

    while x < x2 :
        x = x+1
        if p < 0 :
            p = p + 2 * dy
        else :
            y = y + 1
            p = p + 2 * dy - 2 * dx
        points.append ((x,y))

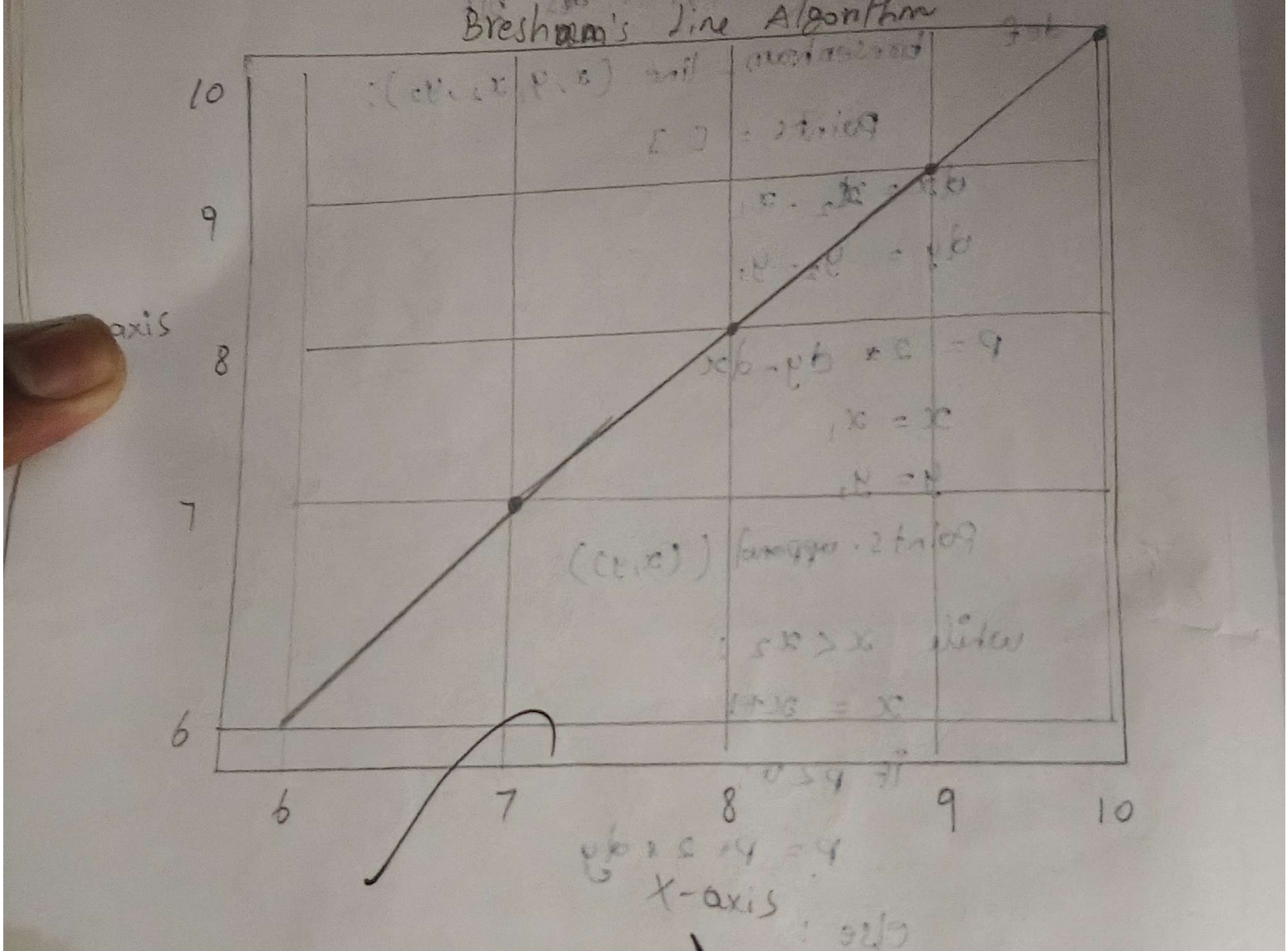
    return points

x1 = int (input ("Enter x1 : "))
y1 = int (input ("Enter y1 : "))
x2 = int (input ("Enter x2 : "))
```

Output:

X: [6, 7, 8, 9, 10]

Y: [6, 7, 8, 9, 10]



o/p verified

21/7/26

```
x-points, y-points - bresenham (x0, y0, xn, yn)
print ("points on the lines:")
for (x,y in zip (x-points_ y-points):
    print (f"({x}, {y})")
plt. figure (figsize = (6,6))
plt. plot (x_points_ y_points, marker = 's',
linestyle = '-', color = 'blue')
plt. title = '_', color = 'blue')
Algorithm (m<1)")
plt. xlable ("x-axis")
plt. ylable ("y-axis")
plt. grid (True)
plt. axis ("equal")
plt. show ()
```

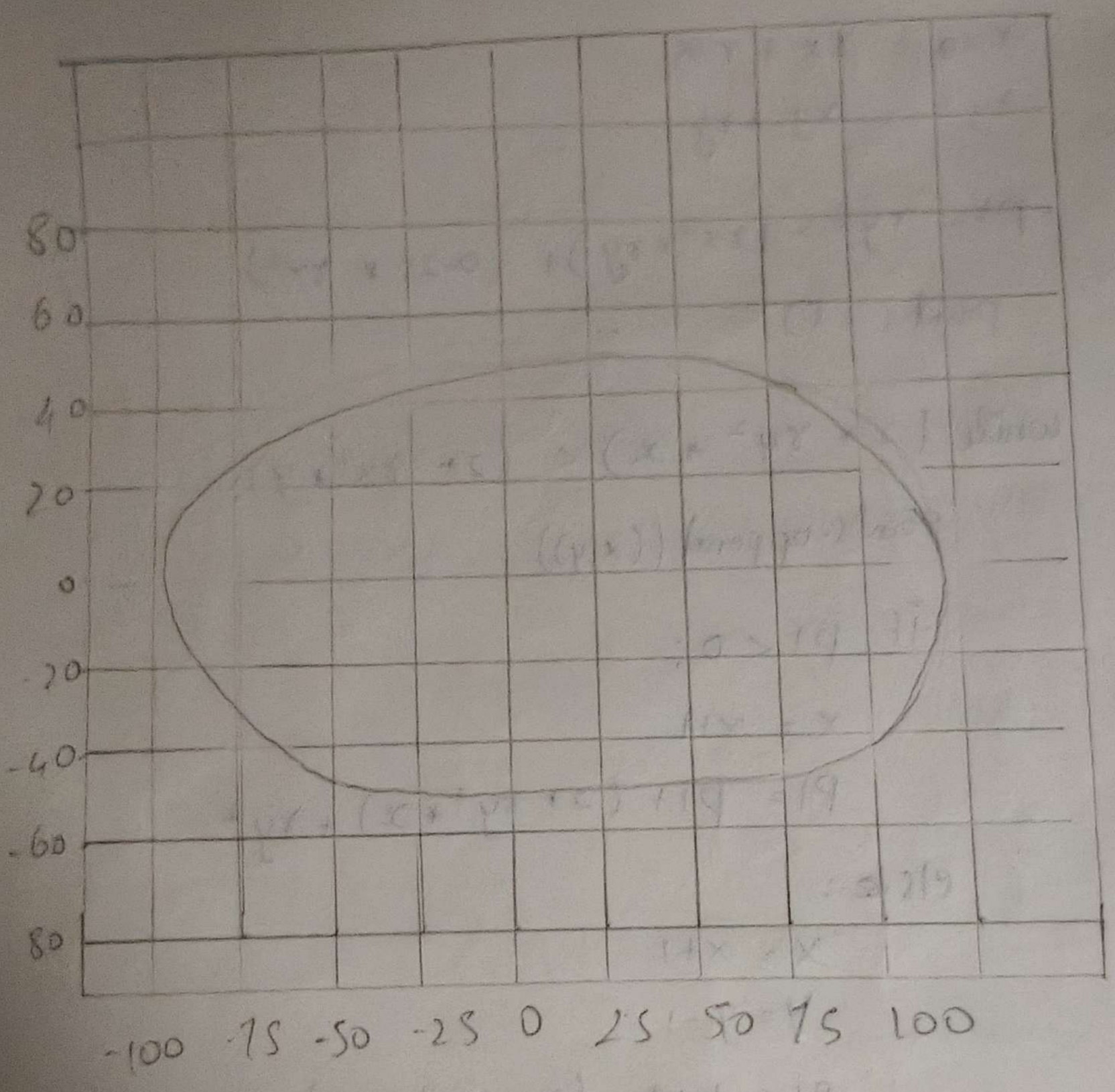
21/7/20

RESULT: The program was executed successfully Bresenham's Line drawing Algorithm for the input where slope |m| < 1.

EXP : 5
21/7/26

BRESENHAM'S LINE DRAWING ALGORITHM FOR ECLIPSE

```
import matplotlib.pyplot as plt
def bresenham_ellipse (xc, yc, rx, ry):
    x = 0
    y = ry
    rx2 = rx * rx
    ry2 = ry * ry
    p1 = ry2 - (rx2 * ry) + (0.25 * rx2)
    points = []
while (2 * ry2 * x) < (2 * rx2 * y):
    points.append ((x,y))
    if p1 < 0:
        x = x+1
        p1 = p1 + (2 * ry2 * x) + ry2
    else:
        x = x+1
        y = y-1
        p1 = p1 + (2 * ry2 * x) - (2 * rx2 * y) + ry2
    p = (ry2 * (x+0.5) * (x+0.5)) + \
        (rx2 * (y-1) * (y-1)) - \ (rx2 * ry2
for x, y in points:
    ellipse_points.append ((xc + x, yc + y))
    ellipse_points.append ((xc - x, yc + y))
    ellipse_points.append ((xc + x, yc - y))
    ellipse_points.append ((xc - x, yc - y))
return ellipse_points
```



Input:

Enter x- coordinate of ellipse centre xc = 50

Enter y- coordinate of ellipse center yc = 50

Major radius of ellipse rx = 40

Major radius of ellipse ry = 20

```
xc = 0
yc = 0
rx = 100
ry = 50

points = bresenham_ellipse (xc, yc, rx, ry)

x_values = [p[0] for p in points]
y_values = [p[1] for p in points]

plt.scatter (x_values, y_values, s = 2)
plt.xlabel ("x-axis")
plt.ylabel ("y-axis")
plt.title ("Bresenham's Ellipse Drawing Algorithm")
plt.axis("equal")
plt.grid()
plt.show()
```

28/7/24

RESULT:

The Bresenham circle Algorithm was Successfully implemented

Exp: 3
21/7/26

BRESENHAM'S LINE DRAWING ALGORITHM FOR CIRCLE

Aim: To implement Brezenham's line drawing algorithm for circle using python program

CODE:

```
import numpy as np
import matplotlib.pyplot as plt

xc = 0
yc = 0 , r = 5

theta = np.linspace(0, 2*np.pi, 50)

circle_x = xc + r * np.cos(theta)
circle_y = yc + r * np.sin(theta)
plt.figure(figsize =(6,6))
plt.plot( circle_x, circle_y color = 'blue',
linestyle = '-', marker ='o', markersize = 5,
linewidth = 2, label = 'connected points')
plt.axis('equal')
plt.grid(True)
plt.legend()
print("In circle Coordinates :")
for x, y in zip(circle_x, circle_y):
    print(f"({x:.2f}, {y:.2f})")
```

28/7/26

Result:

Hence Brezenham's Line drawing Algorithm for circle is drawn & implemented.

Exp: 3
21/7/

Aim:

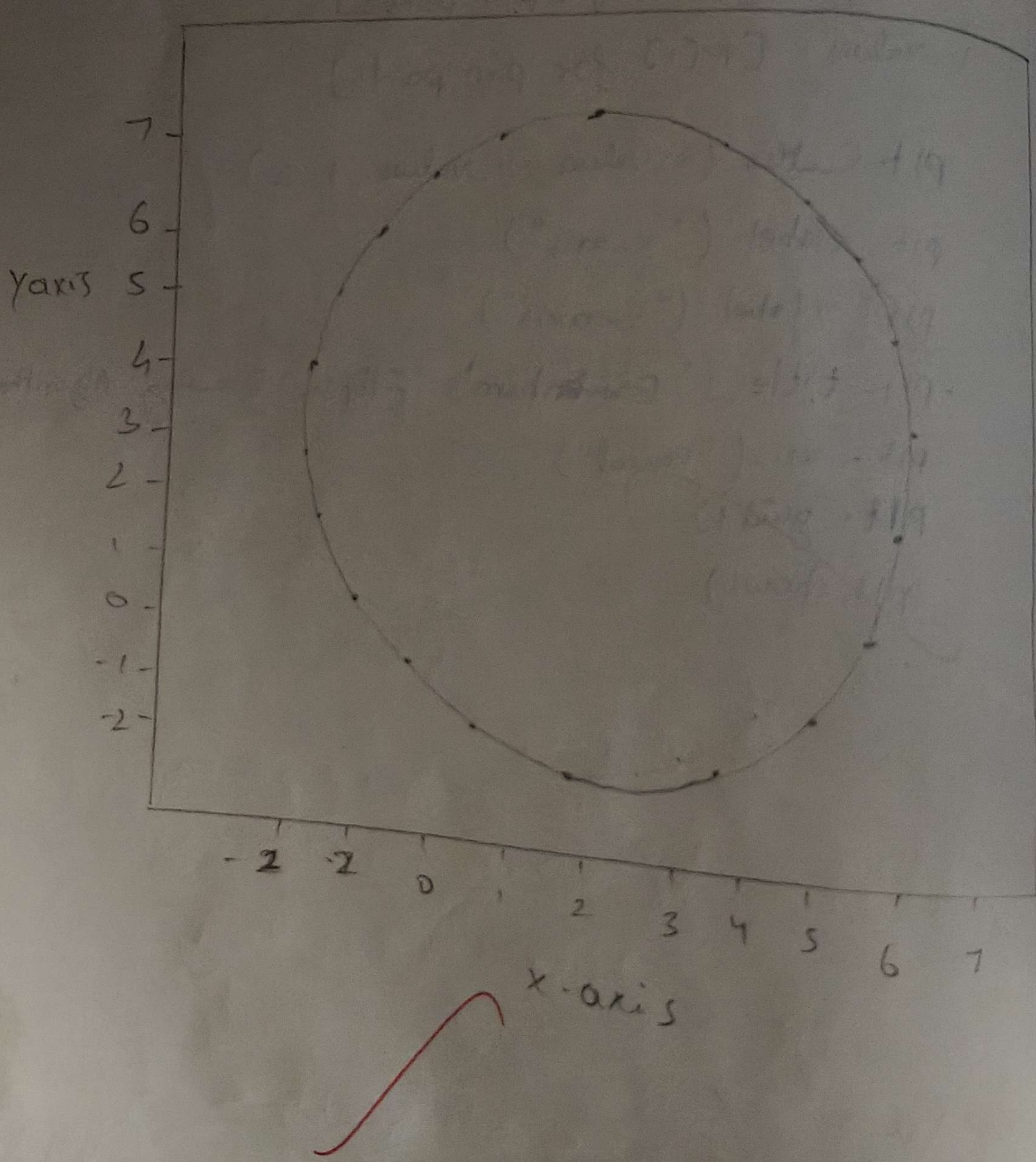
CODE:
im
in

Input: $x_c : 2$

$y_c = 2$

$r = 4$

points : (2,6) (2,-2) (3,-2) (1,-2) (6,2) (-2,2)
(6,2) (-2,2) (6,2) (-2,2) (3,6) (1,6)
(3,-2), (1,-2) (6,3) (4,5) (-2,2)



Ex: 4
Date: 4/8/26

Implementation of DDA Line Drawing Algorithm

Aim: To implement the Digital digital Differential Analyser (DDA) Line Algorithm in Python.

Code:

```
import matplotlib.pyplot as plt

def dda_line_drawing (x0, y0, x1, y1):
    dx = x1 - x0
    dy = y1 - y0

    steps = abs(dx) if abs(dx) > abs(dy) else
            abs(dy)

    x_increment = dx / float (steps)

    x = x0
    y = y0

    points. append ((round (x), round (y)))

    for - in range (int(steps)):
        x+ = x_increment
        y+ = y_increment
        points append ((round (x), round (y)))
```

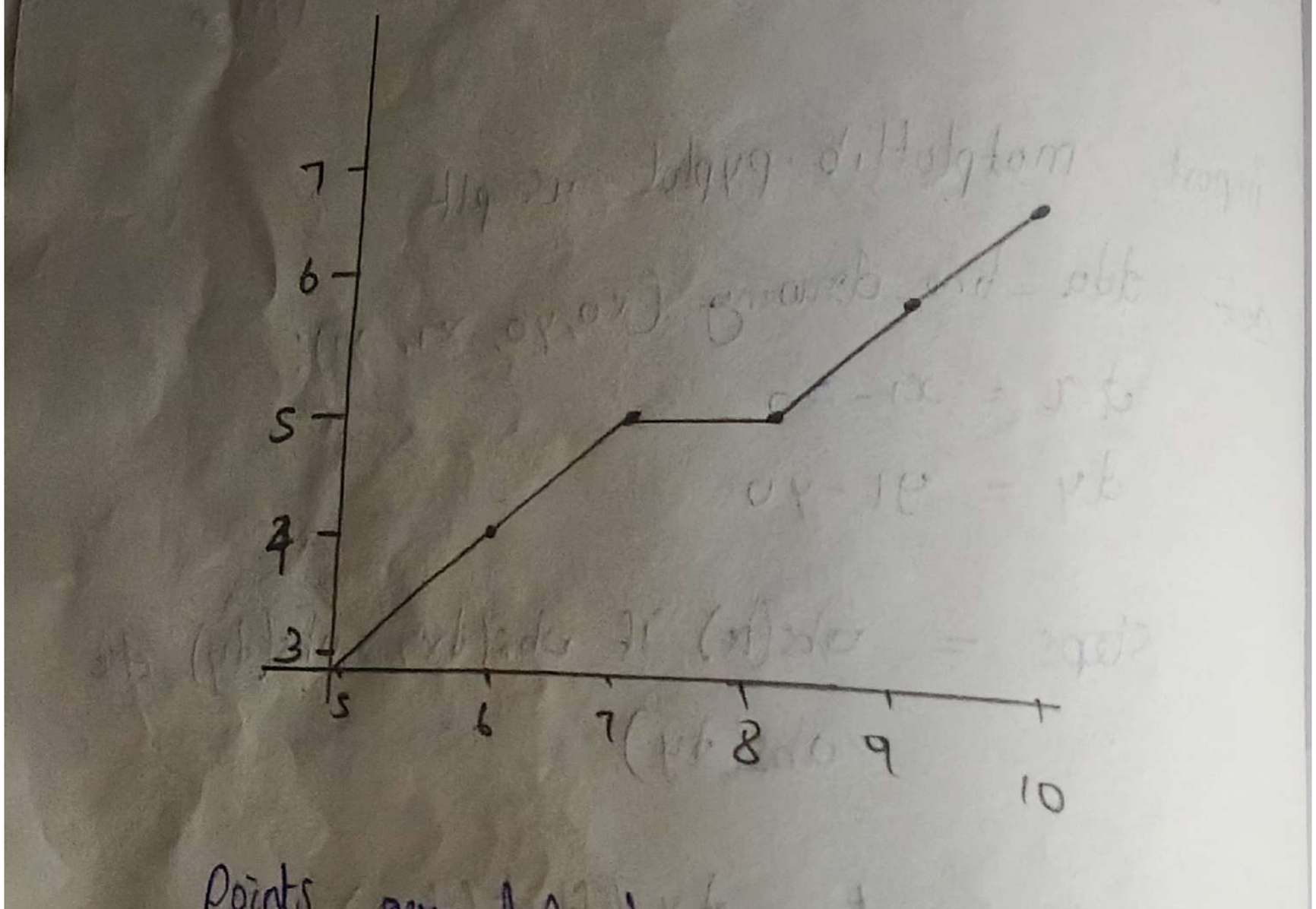
x0 , y0 = 5, 3 } - Start
x1 , y1 = 10, 7 } - End

```
plt.figure(figsize=(6,4))
plt.plot(x_coords, y_coords, 'o-')
plt.xlabel('x-axis')
plt.ylabel('y-axis')
plt.grid(True)
plt.xticks(range(min(x_coords), max(x_coords)+1))
plt.gca().set_aspect('equal', adjustable='box')
plt.show()
print(f"points generated by DDA algorithm:
{line_points}")
```

21/8/26

RESULT:

Hence DDA Algorithm implemented Successfully



Points generated by DDA algorithm:

[(5,3), (6,4), (7,5), (8,5), (9,6), (10,7)]

Exp: 5 Implementation of Drawing an object
Date: 4/8/25 using Bresenham's Algorithm

Aim: To draw an object using
Bresenham's Algorithm

Code:

```
Import matplotlib.pyplot as plt

Character_segements = [ (50, 20, 50, 70),

(40, 70, 60, 70)  (40, 70), (40,) (90) )

(60, 70, 60, 90) , (40, 90, 60, 90)

# Arms  ( 50. 60, 30, 50)  # Left

(50, 60, 70, 50)  # Right

# Legs  (50, 20, 35, 0)  # Left

(50, 20, 65, 0)] # Right

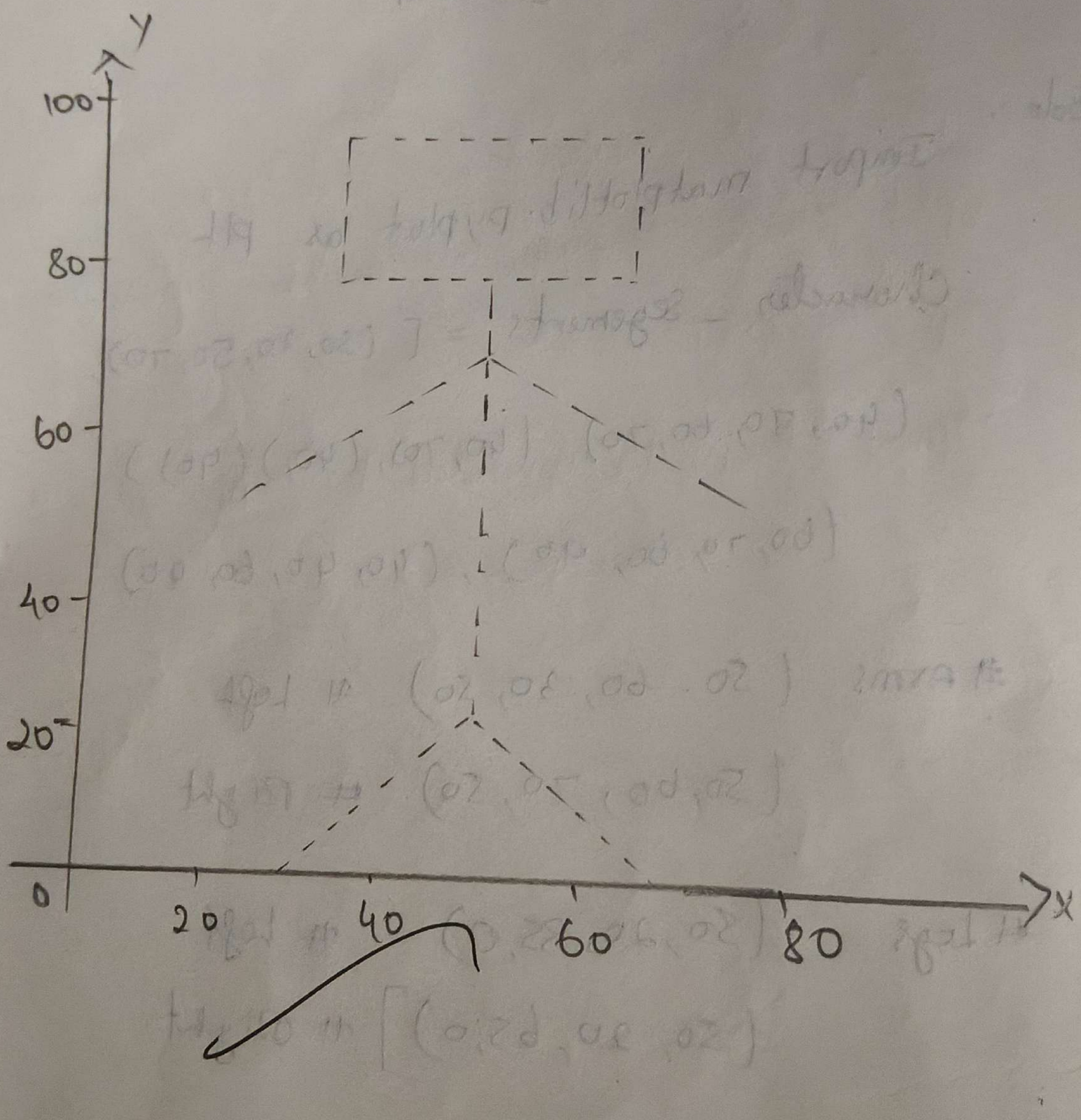
character_pixels = []

for x1, y1, x2, y2 in character
segements:

character_pixels.extend ( bresenham_

line (x1, y1, x2, y2))

xs, ys = zip ( *character_pixels)
```


y
100
80
60
40
20
0
20
40
60
80
x

```
plt.figure (figsize =(8,8))
plt.scatter (xs, ys, s=2, color = 'red')
plt.  xlabel ( 'x - coordinate' )
plt . ylabel ( 'y' - coordinate)
plt . grid (True)
plt.xlim (0, 100)
plt.show()
```

11/8/24

RESULT :

Hence an object is drawn using Bresham Algorithm

Expno: 6
Date: 11/8/26 RECTANGLE USING BRESENHAM'S ALGORITHM

Given: width = 8, height = 5, vertex (2, 2)

Aim: To draw a rectangle using Bresenham's algorithm

Code:

```
import matplotlib.pyplot as plt

def brosenham (x0, y0, x1, y1):
    p = []; dx = abs(x1-x0); dy = abs(y1-y0)
    sx = 1 if x0 < x1 else -1; sy = 1 if
    y0 < y1 else -1
    e = dx - dy
    while 1:
        p.append ((x0, y0))
        if (x0, y0) == (x1, y1) : return p
        e2 = 2 * e

for a, b, c, d in [(x, y, xe, y), (x, ye, xe, ye),
                   (x, y, x, ye), (xe, y, xe, ye
    p = brosenham (a, b, c, d)
    plt.plot (*zip (*p), 'bo-')
plt.grid ()
plt.axis ('equal')
plt.show ()
```

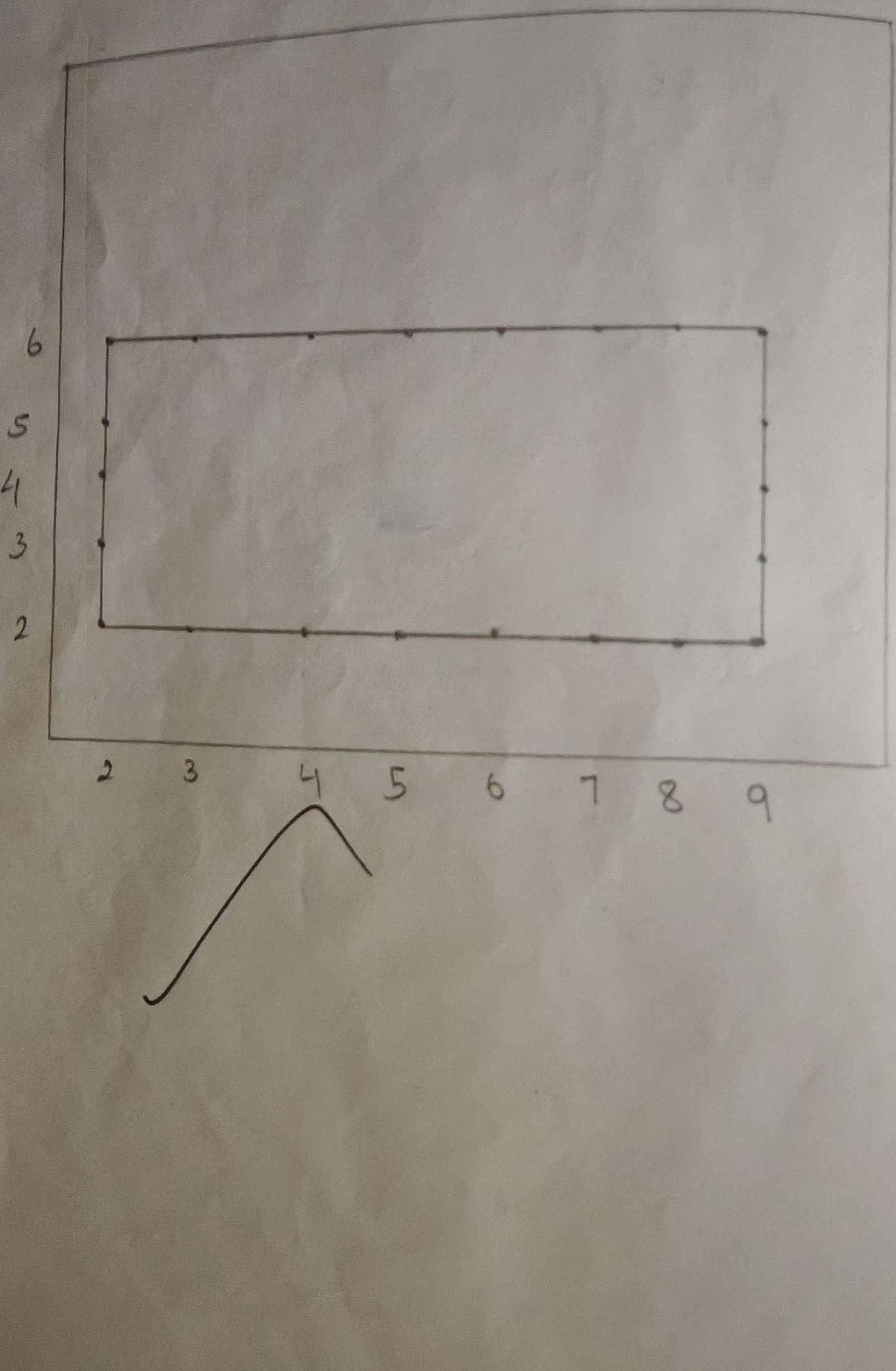
RESULT:

Hence an rectangle is drawn using

Given

Aim:

Output:



Exp no: 7
Date: 11/9/26

Create a traffic signal using mid point Algorithm

Aim: TO implement a traffic signal using mid point Algorithm

Code:

```
import matplotlib.pyplot as plt
import matplotlib.pyplot as patches

def draw_static_light (active_color):
    fig, ax = plt.subplots (figsize = (3,6))
    housing = patches.Rectangle ((1,1), (3,7) L=2,
    red_color = '#ff3333' if active_color == 'red'
    else '#441111'
    yellow_clor = '#33ff3' if active color == 'yellow
    green_color = '#ffcc00' if active color == 'green'

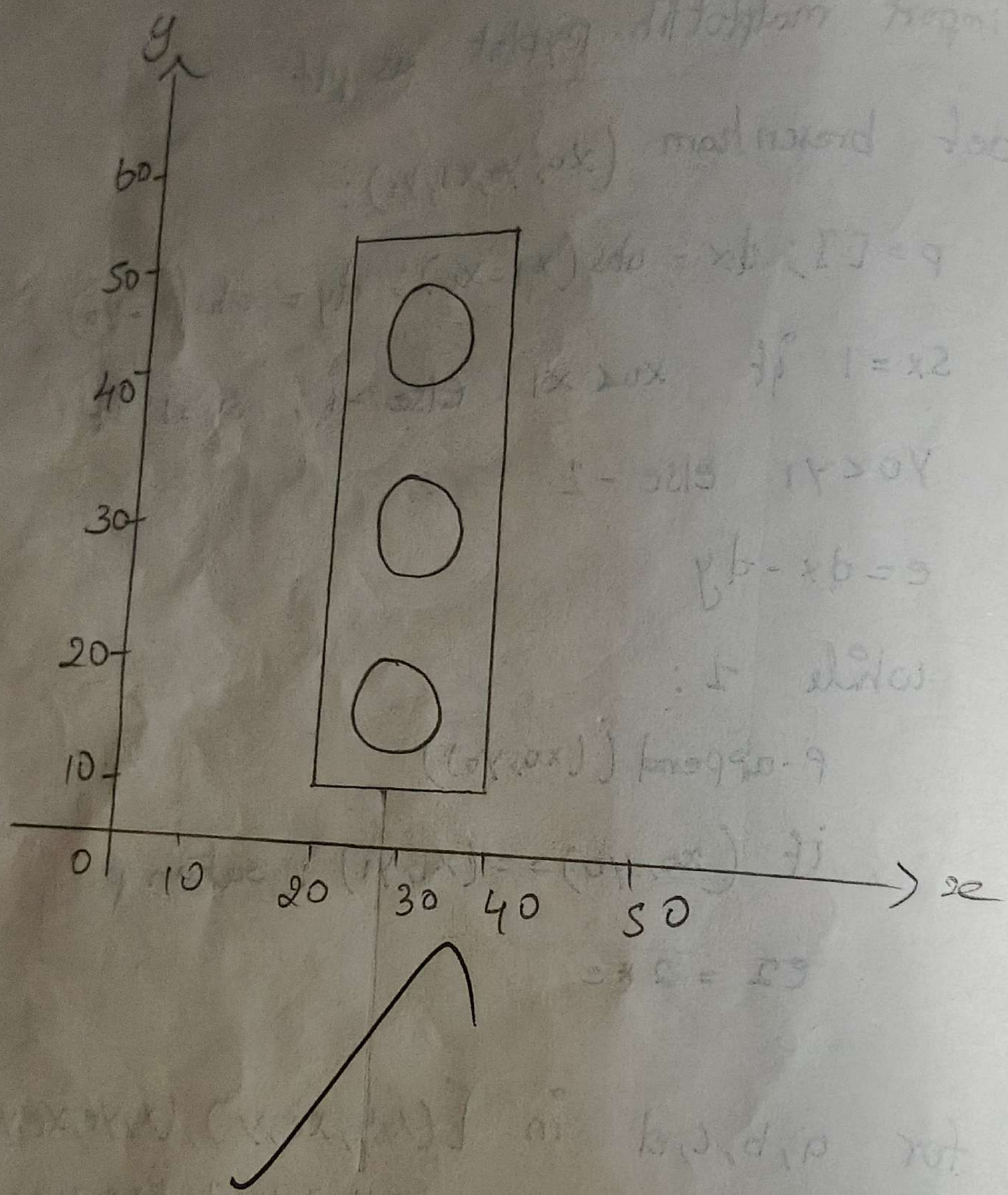
    ax.add_patch (red_circle)
    ax.add_patch (yellow_ circle)
    ax.add_patch (green_ circle)
    ax.set_xlim (0,5)
    ax.set_ylim (0,9)
    ax.axis ('off')
    plt.show()
draw_static_light(active_color = 'red')
```

RESULT:

Exp no: 7
Date: 11/8/26

Aim: To

output:



code:

Expno: 9
Date: 11/8

Draw a smiling face using circle with eyes, nose, mouth

Aim: To implement a smiling face using mid point algorithm

Code:

```
import matplotlib.pyplot as plt
fix, ax = plt.subplots (figsize =(4,4))
ax.add_patch (plt.Circle ((0,0), 1.0, color = 'yellow',
ec ='black', lw=2))
ax.add_patch (plt.Circle ((-0.35, 0.35), 0.12, color ='black'))
ax.add_patch (plt.Circle ((0.35, 0.35), 0.12, color = 'black'))
smile_x = [-0.4, -0.2, 0, 0.2, 0.4]
smile_y = [-0.2, -0.4, -0.45, -0.4, -0.2]
alt.plot (smile_x, smile_y, color = 'black', lw=3,
solid_capstyle ='round')
ax.set_xlim (-1.2, 1.2)
ax.set_ylim (-1.2, 1.2)
ax.axis ('off')
Plt.show
```

RESULT:

Hence an output is obtained.

Expno: 9 Draw

Date: 11/9 with

Aim : To imple
point

Code:

impor

fix,

ax.

ec

ax.

ax.

fig

fig

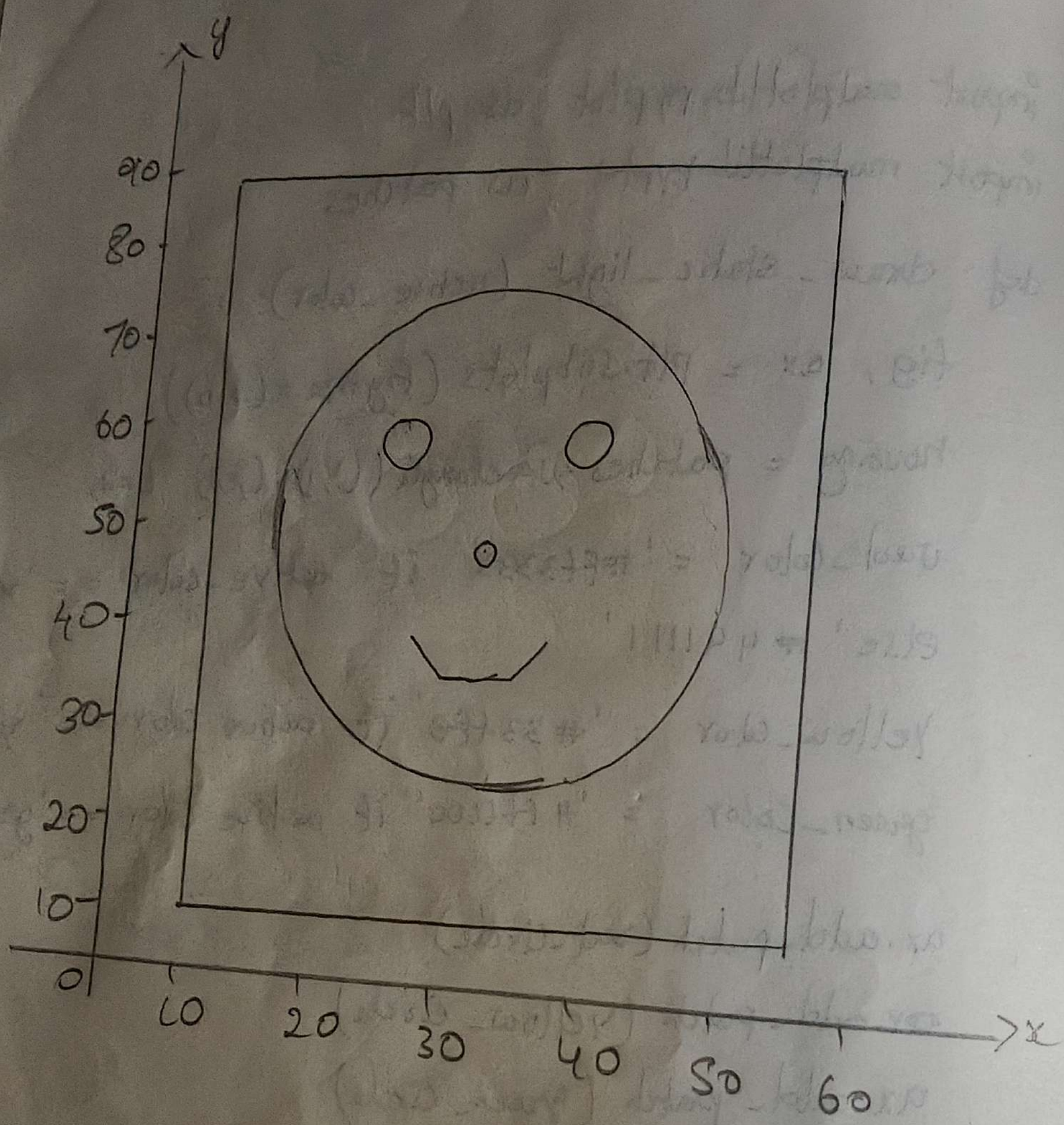
al

8

a

a

output:



EXP - 11

8/9

Aim: To implement scan conversion of polygon & use flood fill algorithm:

Program:

```
Import matplotlib.pyplot as plt

P = [(2,2), (5,8), (9,3), (7,1), (3,1)]

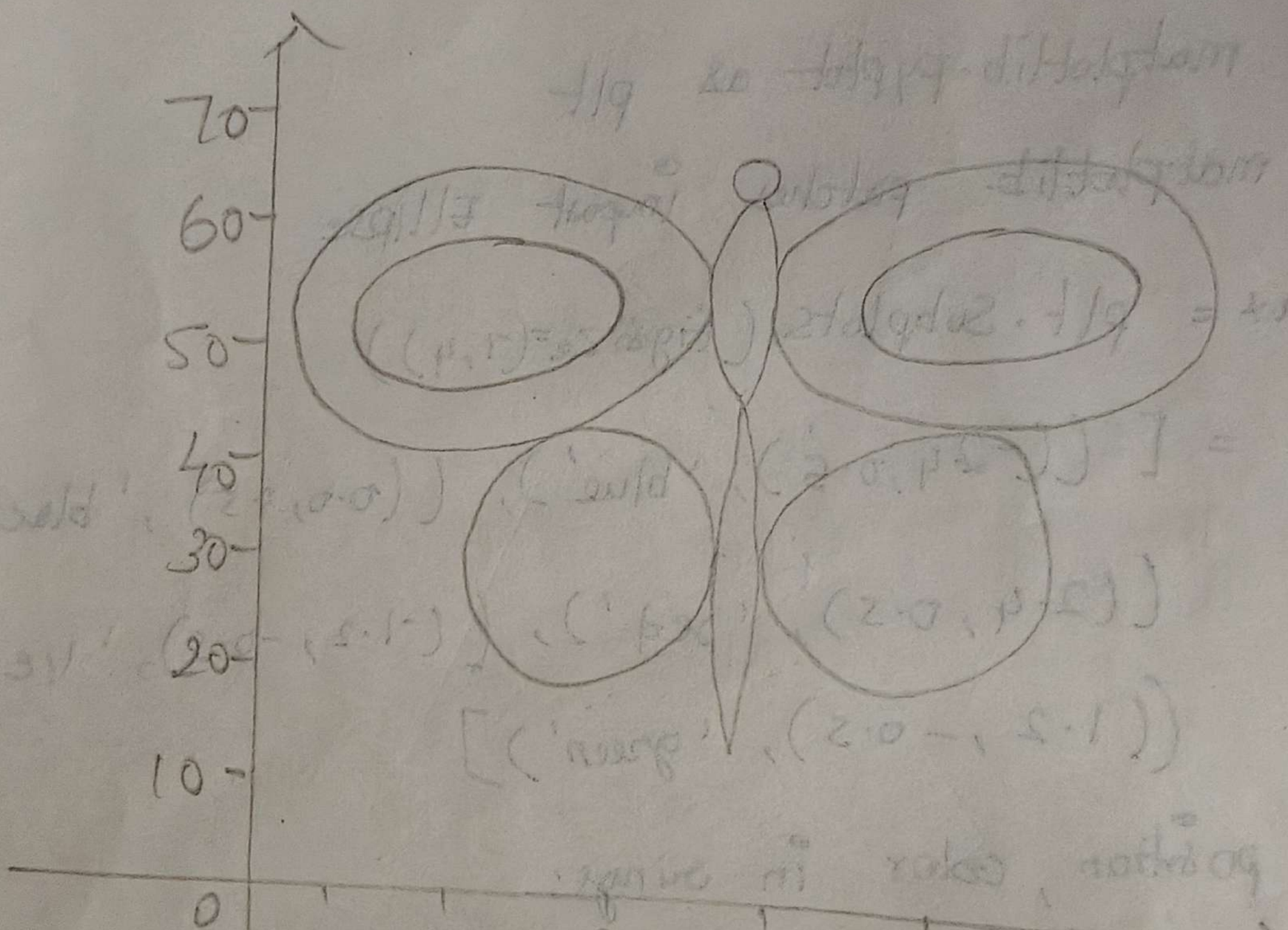
def scan_fill(P):
    ys = range(min(y for x,y in P), max(y for x,y in P)+1)
    for y in ys:
        x = []
        for i in range(len(P)):
            x1, y1 = P[i]
            x2, y2 = P[(i+1)%len(P)]
            if y1 != y2 and min(y1,y2) <= y < max(y1,y2):
                x.append(x1 + (y-y1)*(x2-x1)/(y2-y1))
        x.sort()
        for i in range(0, len(x), 2):
            plt.plot(x[i:i+2], [y,y], 'b-')

x,y = zip(*(P+[P[0]]))
plt.plot(x,y,'k-')
scan_fill(P)
plt.fill(x,y, color = 'red', alpha = 0.1)
plt.axis('equal')
plt.title("scan conversion and flood fill')
plt.show()
```

RESULT:

Hence an output is obtained

output:



Date: 11/8

Exp no: 10 CREATE A BUTTERFLY USING ELLIPSE

Aim:

To draw a Butterfly using Ellipse (mid-point Algorithm)

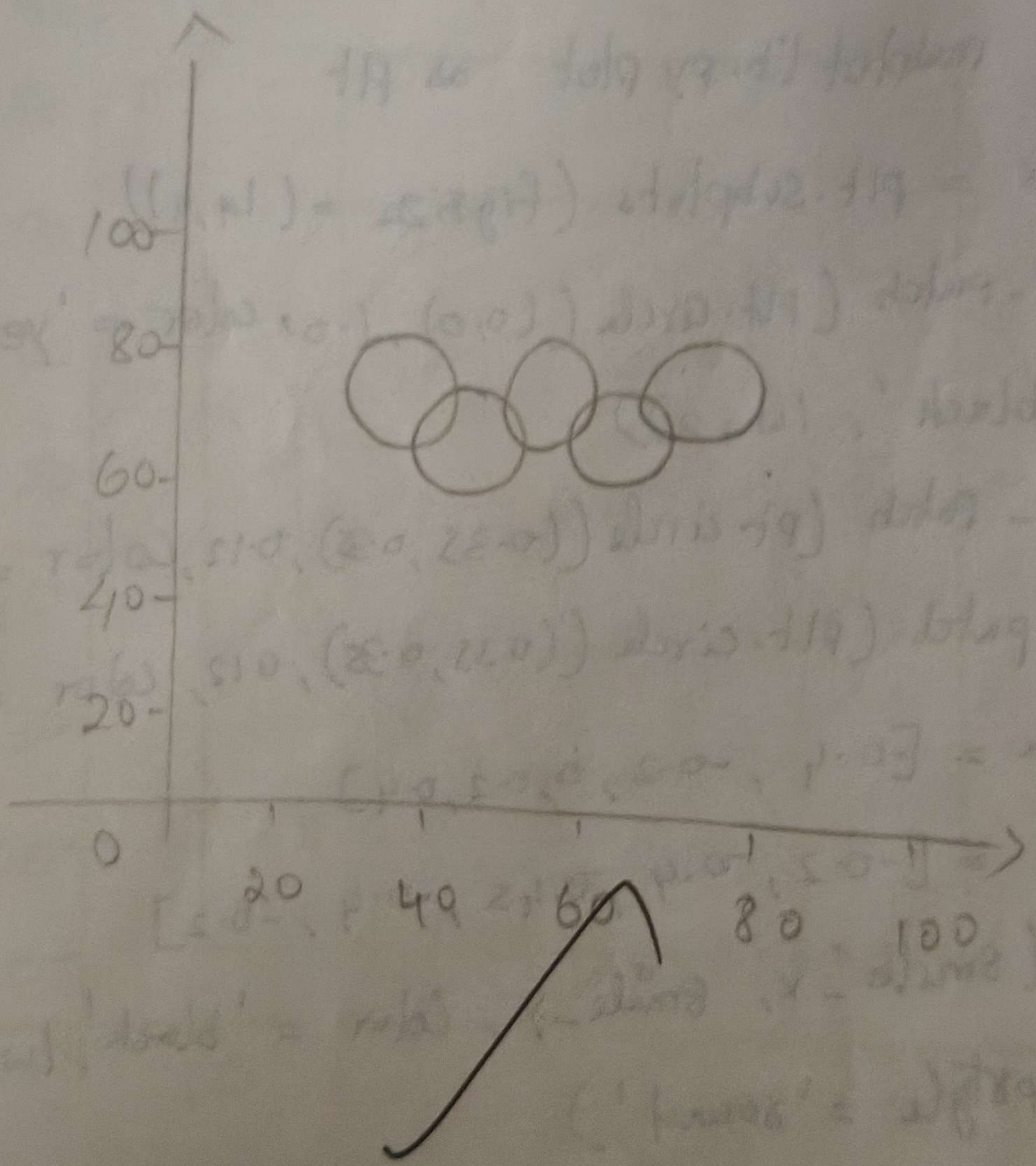
Code:

```
import matplotlib.pyplot as plt
from matplotlib.patches import Ellipse
fig, ax = plt.subplots(figsize=(6,6))
ax.add_patch(Ellipse((-0.6,-0.4), 1.2, 1.6, angle=40,
color='deepskyblue', alpha=0.8))
ax.add_patch(Ellipse((0.9,0.5), 1.8, 2.2, angle=35,
color='darkorange', alpha=0.9))
ax.add_patch(Ellipse((0,0), 0.25, 2.0, color='saddlebrown'))
ax.add_patch(Ellipse((0,1.1), 0.35, 0.35, color='saddlebrown'))
ax.set_xlim(-2.5, 2.5)
ax.set_aspect('equal')
ax.axis('off')
plt.show()
```

RESULT:

Hence an output is obtained

Output :



Exp no: 9
Date: 11/8

DRAW A OLYMPIC LOGO USING ELLIPSE

Aim:

To implement a olypic Logo using mid point algorithm

Code:

```
import matplotlib.pyplot as plt
from matplotlib.patches import Ellipse

fig, ax = plt.subplots(figsize=(7,4))

rings = [((-2.4,0.5), 'blue'), ((0.0,0.5), 'black'),
         ((2.4,0.5), 'red'), ((-1.2,-0.5), 'yellow')
         ((1.2,-0.5), 'green')]

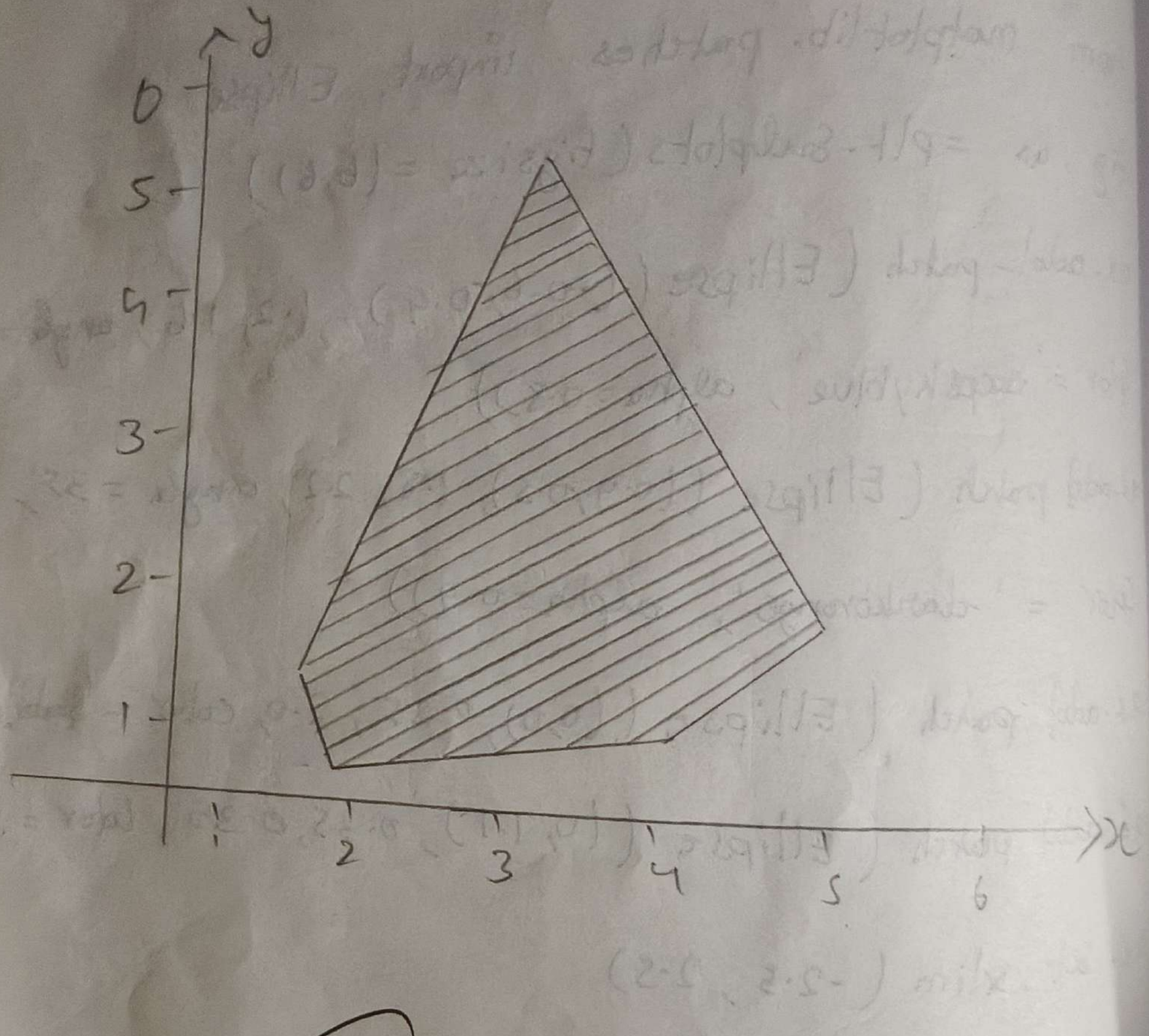
for position, color in rings:
    ax.add_patch(Ellipse(position, 2.0, 2.0, color=color,
    fill=False, lw=6))

ax.set_xlim(-4,4)
ax.set_ylim(-2,2)
ax.set_aspect('equal')
ax.axis('off')
plt.show()
```

RESULT:

Hence an output is obtained

output:



Ex no: 14
Date:

Create and Render 3D objects using polygons and Apply Basic Color and Shading Techniques.

Aim: To create and render 3D objects

* Flat Shading
* Basic color technique

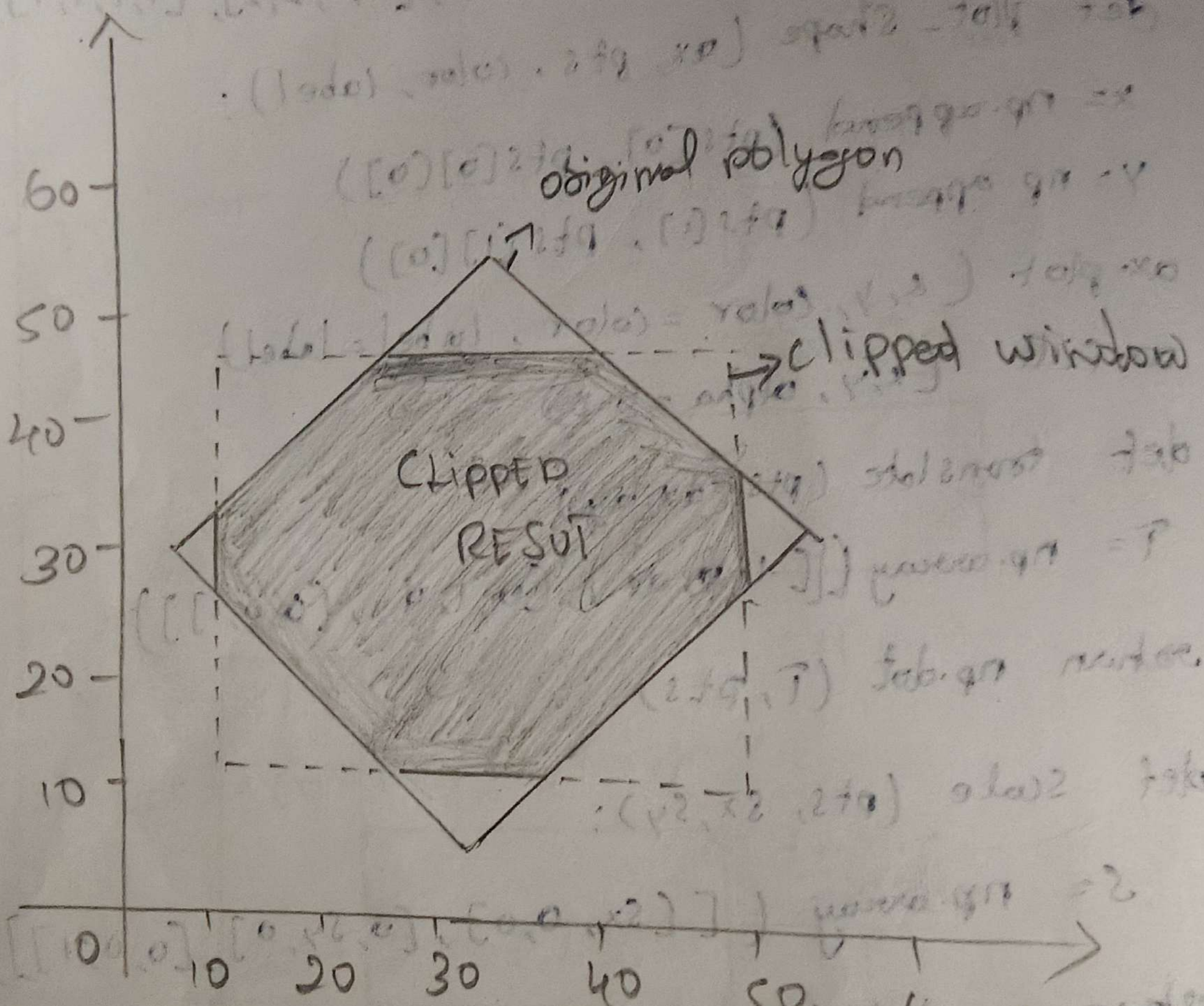
Program:

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import
Poly3DCollections
import numpy as np
vertices = np.array([[0,0,0], [1,0,0], [1,1,0], [0,1,0],
[0,0,1], [1,0,1], [1,1,1], [0,1,1]])
faces = [[vertices[j] for j in [0,1,2,3]],
         [vertices[j] for j in [4,5,6,7]],
         [vertices[j] for j in [0,1,5,4]],
         [vertices[j] for j in [2,3,7,6]],
         [vertices[j] for j in [4,7,3,0]]]
```

RESULT:

Hence the output is verified

output


original polygon
clipped window
CLIPPED RESULT
60
50
40
30
20
10
0
10
20
30
40
50

22/9 EXP: 13

POLYGON CLIPPING USING SUTHERLAND HODGMAN ALGORITHM

Aim:

To clip the window using polygon clipping by sutherland Hodgman Algorithm

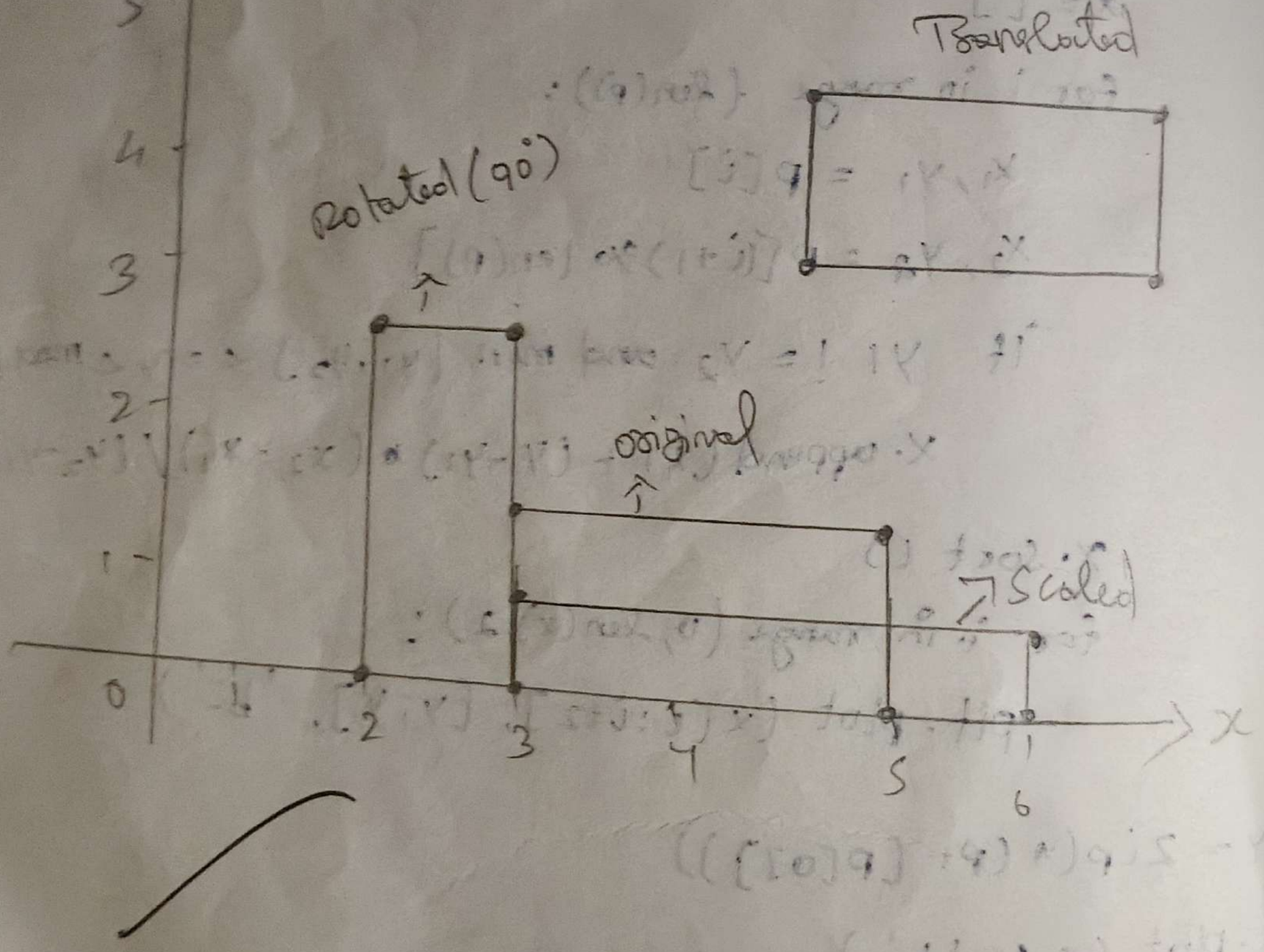
Code:

```
! pip install -q shapely matplotlib
import matplotlib.pyplot as plt
from shapely.geometry import polygon
poly_subj = polygon ( [(100,150), (200,250), (300,150),
                       (200,50)])
poly_subj = polygon ( [(120,80), (120,200), (280,200), (280,80)])
poly_resv = poly_subj.intersection (poly_clip)
fig, ax = plt.subplots (figsize =(6,6))
ax.plot (*poly_subj.exterior.xy, 'b--', label)
ax.set title ('Sutherland-Hodgman clipping)
ax.legend ()
ax.grid (True)
plt.show()
Print ("clipped vertices:", list (poly_resv.exterior.coords)[:-1])
```

RESULT:

Hence output is obtained

output



Exp
Date : 15/08

Aim: To

import
import
rect =

def plo
x= np.
y= np.
ax.plot
ax.fill
def t
T= n

return

def s

s=

return

t_rec

t_sc

x_sc

Plot_

plot_

plot_

Plot_

RESULT

Exp no: 12
Date: 15/08

Implementation of 2D Shapes and TO Perform Basic Transformations

Aim:
To draw a 2D shape & perform basic Transformation

```
import matplotlib.pyplot as plt
import numpy as np
rect = np.array([[0,4,4,0],[0,0,2,2],[1,1,1,1]])
def plot_shape(ax, pts, color, label):
    x= np.append(pts[0], pts[0][0])
    y= np.append(pts[1], pts[1][0])
ax.plot(x, y, color = color, label = label)
ax.fill(x,y, alpha =0.2)
def translate(pts, dx, dy):
    T= np.array([[1,0,dx],[0,1,dy],[0,0,1]])
return np.dot(T, pts)
def scale(pts, sx, sy):
    S= np.array([[sx,0,0],[0,sy,0],[0,0,1]])
return np.dot(S, pts)
t_rect = translate(rect, 3, 2)
s_rect = scale(3.5, 0.5)
r_rect = rotate(90)
Plot_shape(ax, rect, "blue", "Original Rect")
plot_shape(ax, t_rect, "green", "Translated")
plot_shape(ax, s_rect, "orange", "scaled")
plot_shape(ax, r_rect, "purple", "Rotated (90°)")
```

RESULT:

Hence an output is obtained.

Ex:17

Capture Video/Audio from webcam or Microphone and Display on multimedia Interface

Aim: To develop a program that Captures

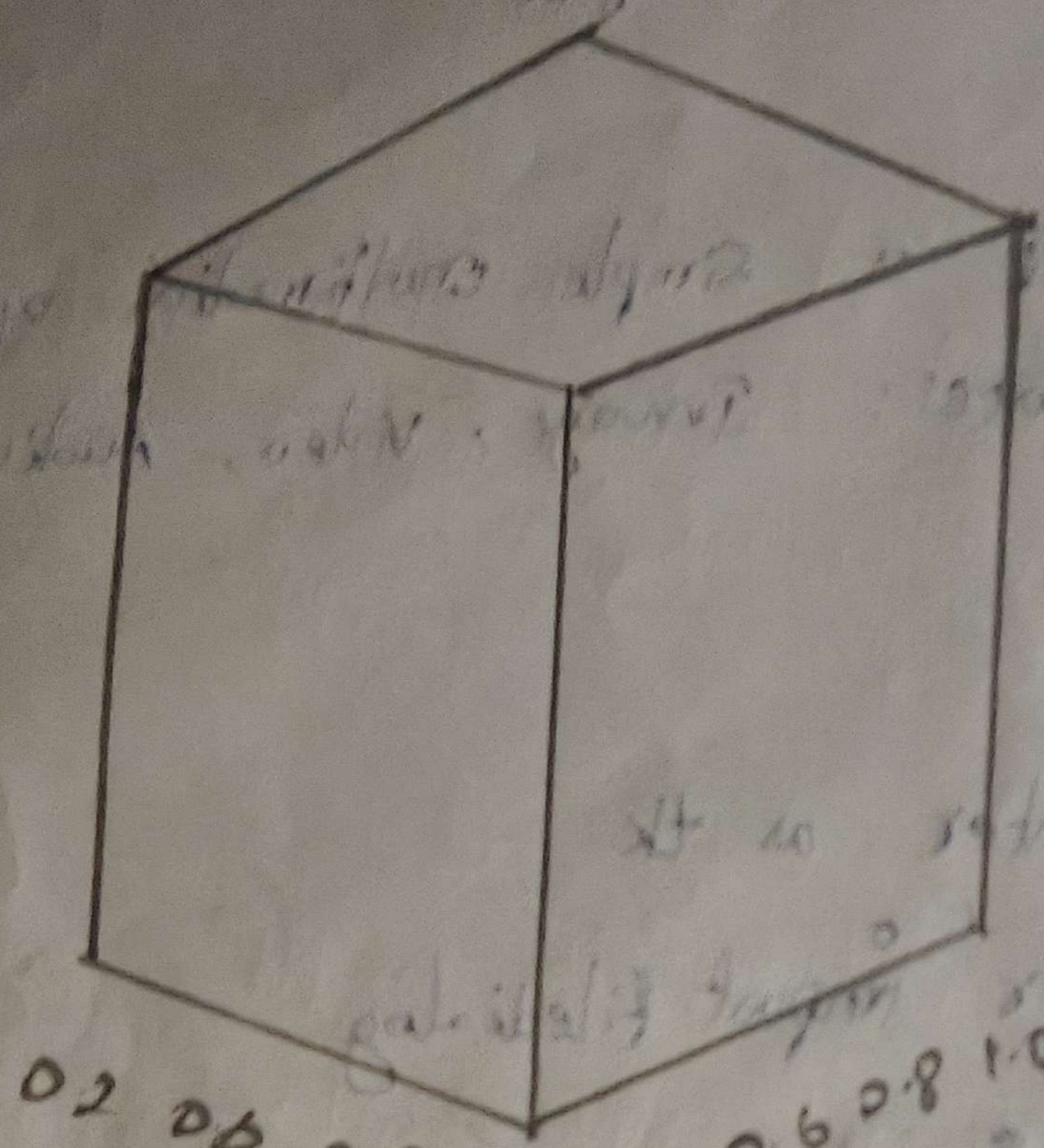
- Live video from the webcam
- Live audio from the Microphone

Program:

```
import cv2
import sounddevice as sd
from scipy.io.wavfile import write
import threading
import tkinter as tk

def record_audio():
duration = 5
fs = 44100
audio = sd.rec(int(duration * fs), samplerate=fs, channels=2)
sd.wait()

def capture_video()
cap = cv2.VideoCapture(0)
  while True:
    ret, frame = cap.read()
    if not ret:
      break
```



3D cube with flat shading

Ex no:15
Date:

3D Transmations on Basic objects (cube. pyramid)

Aim:

To write a program that allows the user to perform 3D transformations.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Line3DCollection

def draw_edges(ax, vertices, edges, color='b'):
    lines = [(vertices[start], vertices[end])

def translation_matrix(tx, ty, tz):
    return np.array([[1,0,0,tx],[0,1,0,ty],])

def scaling_matrix(sx, sy, sz):
    return np.array([[sx,0,0,0]])

def rotation_matrix(angle):
    rad = np.radians(angle)
    return np.array([[[np.cos(rad), np.sin(rad),0,0],

ax.set_box_aspect([1,1,1])
plt.show()
```

RESULT:

Hence the output is verified

Ex: 15

Create a Multimedia Application that Integrates Images, Sound, and Video in a simple user Interface

Aim: To develop a simple Multimedia application that integrates: Image, Video, Audio.

Program:

```
import tkinter as tk
from tkinter import filedialog
from PIL import Image, ImageTk
from playsound import playsound
import cv2
import threading
panel = tk.Label(window)
Panel.pack()
btn_img.pack(pady=5)
btn_audio.pack(pady=5)
window.mainloop()
window.tk.Tk()
window.title("multimedia App")
```

Result:

Hence an output is verified

Create an Application to send and
Receive multimedia Messages (Text, Image, Audio)

Aim: To develop a basic chat-like application

Program:

```
import tkinter as tk
from tkinter import filedialog, scrolledtext
from PIL import Image, ImageTk
from playsound import playsound

def send_text():
    msg = text_entry.get()

    if msg:
        chat_box.insert(tk.END, f"You: {msg}\n")
        text_entry.delete(0, tk.END)

def send_image():
    path = filedialog.askopenfilename(filetypes=[("Images")]
    if path:
        chat_box.insert(tk.END, f"You sent an image: {path}\n")
        threading.Thread(target=lambda: playsound(path)).start()

window = tk.Tk()
window.title("Multimedia")
window.geometry("400x500")
```

```
Label_Frame = tk.Frame(window)
btn_text.pack(side=tk.LEFT)
btn_image.pack(side = tk.LEFT)
btn_audio.pack(side = tk.LEFT)
window.mainloop()
```

Result:

Hence an output is verified